

Tesla Financial Analysis Project — Complete Workflow & Code Reference

This guide explains the full project: where data comes from, how each analysis is computed, how results flow to the web dashboard, Excel workbook, and local reports, and what each code file does.

1. Project Overview

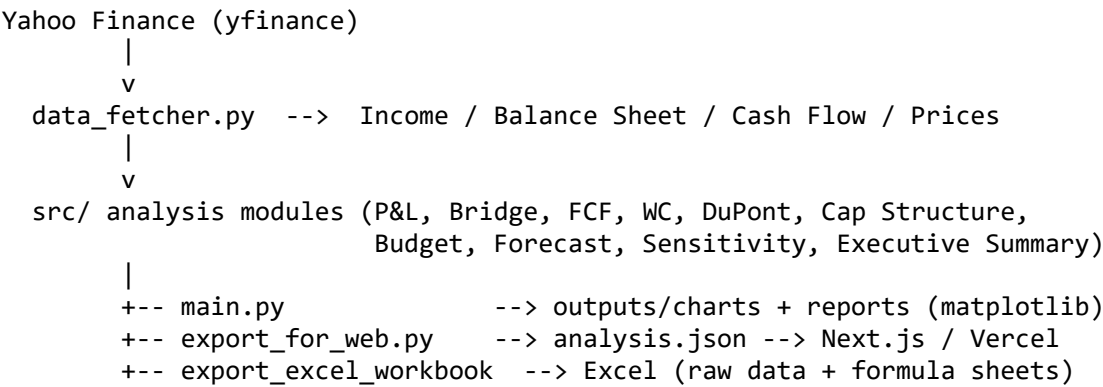
This is a **Corporate Finance / FP&A (Financial Planning & Analysis)** portfolio project using Tesla (TSLA) as the example. It demonstrates an end-to-end pipeline from public financial statements to an interactive dashboard and a formula-driven Excel learning workbook.

1.1 Three Output Channels

Channel	Entry Command	Output	Purpose
Local pipeline	python main.py	outputs/charts/*.png out-puts/reports/*.csv	Run locally; static charts and CSV reports
Web dashboard	python scripts/export_for_web.py	web/data/analysis.json -> Vercel	Public interactive dashboard
Excel workbook	python scripts/export_excel_workbook.py	Desktop .xlsx	FP&A modeling practice with Excel formulas

All three channels **share the same src/ analysis modules**; only the export format differs.

1.2 Architecture



1.3 Directory Structure

```

FInancial Analysis Project/
|-- main.py                # Local 5-step pipeline + matplotlib charts
|-- requirements.txt
|-- src/                   # Core analysis logic
|   |-- data_fetcher.py    # Yahoo Finance fetch + normalization
|   |-- pl_analysis.py     # P&L and margins
```

```
| | -- margin_bridge.py          # Net income bridge
| | -- cash_flow_analysis.py    # OCF, FCF, cash quality
| | -- working_capital.py      # DSO, DIO, DPO, CCC
| | -- dupont_analysis.py      # ROE decomposition
| | -- capital_structure.py    # Leverage and coverage
| | -- budgeting.py           # Budget and variance
| | -- forecasting.py         # Time-series forecast and scenarios
| | -- sensitivity.py          # One-way, two-way, tornado
| | -- executive_summary.py    # Narrative report
| | `-- visualization.py      # matplotlib (main.py only)
|-- scripts/
| | -- export_for_web.py
| | -- export_excel_workbook.py
| | `-- excel_formula_builder.py
|-- web/                          # Next.js 15 (Vercel root = web)
| | -- app/page.tsx
| | -- components/charts.tsx
| | `-- data/analysis.json
`-- outputs/                      # main.py results
```

2. Data Layer: data_fetcher.py

2.1 Responsibility

Fetches from Yahoo Finance:

- Income statement (income_stmt)
- Balance sheet (balance_sheet)
- Cash flow statement (cash_flow)
- Price history (price_history)
- Company info (info)

2.2 Normalization: _normalize_statement

Yahoo raw format: **rows = line items, columns = dates.**

Project standard format:

```
index = Year (2021, 2022, ...)
columns = line items (Total Revenue, Net Income, ...)
```

Steps:

1. Convert column headers from datetime to **integer year**
2. **Transpose** (.T) so years become the index
3. Sort by year

All src/ modules assume: **DataFrame index = fiscal years, columns = line items.**

```
out.columns = pd.to_datetime(out.columns).year
out = out.T.sort_index()
out.index.name = "Year"
```

3. Analysis Modules (src/)

3.1 P&L Analysis — pl_analysis.py

Class: PLAnalyzer(income_stmt)

Method	Output	Logic
margin_analysis()	Gross, operating, net, EBITDA margins (%)	line item / Total Revenue * 100
yoy_growth()	YoY % change per line item	pct_change() * 100
expense_breakdown()	COGS, OpEx, R&D, SG&A as % of revenue	expense / Total Revenue * 100
summary_table()	Values + growth + margins combined	Join of the above

3.2 Net Income Bridge — margin_bridge.py

Class: MarginBridgeAnalyzer(income_stmt)

Method: yoy_bridge() — explains year-over-year net income change.

For each pair (prev -> curr):

Bridge item	Meaning
Revenue Volume	(Rev_curr - Rev_prev) * (GP_prev / Rev_prev)
Margin / Mix	Rev_curr * (GM_curr - GM_prev)
OpEx Change	(OI_curr - OI_prev) - (GP_curr - GP_prev)
Below-the-Line	(NI_curr - NI_prev) - (OI_curr - OI_prev)
Total delta Net Income	NI_curr - NI_prev

3.3 Cash Flow — cash_flow_analysis.py

Class: CashFlowAnalyzer(income_stmt, cash_flow, balance_sheet)

Metric	Logic
Operating Cash Flow	From cash flow statement
Capital Expenditure	abs(Capital Expenditure)
Free Cash Flow	FCF field, or OCF - CapEx if FCF is zero
FCF Margin %	FCF / Revenue * 100
CapEx % of Revenue	CapEx / Revenue * 100
OCF / Net Income	Earnings quality ratio

3.4 Working Capital — working_capital.py

Class: WorkingCapitalAnalyzer(income_stmt, balance_sheet)

Metric	Formula
DSO	Accounts Receivable / Revenue * 365
DIO	Inventory / COGS * 365
DPO	Accounts Payable / COGS * 365
CCC	DSO + DIO - DPO

Metric	Formula
Change in WC	Current year WC minus prior year WC

3.5 DuPont Analysis — dupont_analysis.py

Class: DuPontAnalyzer(income_stmt, balance_sheet)

Three-factor ROE:

ROE = Net Margin x Asset Turnover x Equity Multiplier
 = (NI/Revenue) x (Revenue/Total Assets) x (Total Assets/Equity)

Also outputs ROA and ROIC (EBIT / Invested Capital).

3.6 Capital Structure — capital_structure.py

Metric	Formula
Net Debt	Total Debt - Cash
Debt / Equity	Total Debt / Stockholders Equity
Net Debt / EBITDA	Net Debt / EBITDA
Interest Coverage	Operating Income / abs(Interest Expense)

3.7 Budgeting — budgeting.py

Class: BudgetAnalyzer(income_stmt)

create_budget(base_year, growth_assumptions, years_forward) Forward projection from base-year actuals:

Budget_year_n = Base * (1 + growth_rate)^n

Chain:

1. Revenue, COGS, OpEx grown at respective rates
2. Gross Profit = Revenue - COGS
3. Operating Income = GP - OpEx
4. Net Income = OI * 0.75 (simplified tax)

Default growth: historical **CAGR** if not specified.

Current hard-coded assumptions (in export scripts):

```
growth_assumptions={"Total Revenue": 0.12, "Operating Expense": 0.08}
base_year = income_stmt.index[-2]
latest_year = income_stmt.index[-1]
```

variance_analysis(budget, actual_year)

```
Variance = Actual - Budget
Variance % = Variance / Budget * 100
Status = Favorable (>= 0) / Unfavorable (< 0)
```

budget_summary(base_year, years_forward) Historical actuals + forward budget rows for charts.

3.8 Forecasting — forecasting.py

Class: FinancialForecaster(income_stmt)

forecast_metric(metric, periods=3, method="exponential")

- Historical years: Type = "Actual"
- Future years: Type = "Forecast"

Methods:

1. **exponential** — Holt-Winters (statsmodels), fallback to CAGR
2. **linear** — linear regression extrapolation
3. **cagr** — compound annual growth

scenario_forecast(metric, periods=3) From latest revenue, compound by scenario rate:

Scenario	Default annual growth
Bear	-5%
Base	+10%
Bull	+25%

3.9 Sensitivity — sensitivity.py

Class: SensitivityAnalyzer(income_stmt) — uses **latest fiscal year** as base.

one_way_sensitivity(driver, driver_changes, target) Driver varied +/- 5% to 20%; dependent lines cascade:

- Revenue change -> COGS scales proportionally -> GP -> OI
- Net income scaled by OI * (base_NI / base_OI)

two_way_sensitivity(driver_x, driver_y) Matrix: Revenue x OpEx combinations vs Net Income.

tornado_data(drivers, change_pct=0.10) Each driver +/- 10%; ranked by impact range for tornado chart.

3.10 Executive Summary — executive_summary.py

Class: ExecutiveSummaryGenerator — formats computed results into **English narrative sections** (no recalculation).

Used by web #summary and Excel sheet 21_Executive_Summary.

3.11 Visualization — visualization.py

Used only by **main.py** (matplotlib PNGs in outputs/charts/). The web app uses **Recharts** instead.

4. Export Pipelines

4.1 main.py — Local Pipeline (Core FP&A)

```
[1/5] data_fetcher      -> CSV in outputs/data/
[2/5] pl_analysis       -> margins, growth + charts
[3/5] budgeting         -> budget, variance + chart
[4/5] forecasting       -> revenue forecast, scenarios + charts
[5/5] sensitivity       -> one-way, two-way, tornado + charts
      + text executive_summary.txt
```

Note: main.py does **not** include Corp Fin modules (FCF, DuPont, WC). Use export_for_web.py for the full set.

4.2 scripts/export_for_web.py — Web JSON

Flow:

1. Fetch all statements via FinancialDataFetcher
2. Run all analysis classes
3. Build payload dict
4. Write web/public/data/analysis.json
5. Copy to web/data/analysis.json (Next.js import)

JSON top-level keys:

Key	Content
company	Name, sector, market cap
highlights	KPI cards: revenue, FCF, ROE, CCC, YoY
pl	incomeStatement, margins, growth, expenseBreakdown
budget	summary (with fiscalYear/Type/label), variance
forecast	revenue, multiMetric, scenarios
sensitivity	oneWay, twoWay, tornado
corpFin	cashFlow, workingCapital, dupont, capitalStructure, marginBridge
summary	executive summary sections
stockPrice	Last 252 trading days
generatedAt	ISO timestamp

Helpers:

Function	Role
_df_to_records(df)	Year index -> { year: 2024, ... }
_series_to_chart(df, cols)	Selected columns for chart series
_budget_summary_records(df)	Adds fiscalYear, label, Type for budget chart

4.3 scripts/export_excel_workbook.py — Excel Workbook

Phase 1 — pandas (values only):

- 00_Guide, 00_Pipeline — instructions
- 01_Company_Info
- 02_Income_Raw, 03_Balance_Raw, 04_CashFlow_Raw — vertical layout (rows = line items, cols = years)

- 05_Price_1Y

Phase 2 — openpyxl formulas (excel_formula_builder.py):

- 15a_Assumptions — editable growth and scenario inputs
- 06 through 20 — **Excel formulas** referencing sheets 02-04 and 15a
- 21_Executive_Summary — Python-generated narrative

Formula reference example (gross margin):

```
=IFERROR('02_Income_Raw'!F5/'02_Income_Raw'!F2*100,"")
```

Edit assumptions on 15a_Assumptions to recalc budget, forecast, scenarios, and sensitivity sheets.

5. Web Application (web/)

5.1 Stack

- Next.js 15 (App Router)
- React + TypeScript
- Tailwind CSS (dark theme)
- Recharts (interactive charts)

5.2 Data Loading

```
// web/lib/data.ts
import raw from "@data/analysis.json";
export const analysisData = raw as AnalysisData;
```

JSON is **bundled at build time**; no runtime API.

5.3 Page Sections — web/app/page.tsx

Section ID	Content	Charts
#overview	KPI cards	—
#pl	P&L	RevenueProfit, Margin, Growth, ExpenseBreakdown, MarginBridge
#cashflow	Cash flow	FCFChart + metrics table
#corpfin	Corp fin	DuPont, CCC + capital structure table
#budget	Budget	BudgetVsActual + variance table
#forecast	Forecast	ForecastChart, ScenarioChart
#sensitivity	Sensitivity	Tornado, OneWay + driver ranking
#summary	Narrative report	Text cards
#methodology	How it was built	Info cards

5.4 Charts — web/components/charts.tsx

Each exported component takes a data prop from analysis.json.

Notable: BudgetVsActualChart uses BudgetPoint typing; Actual bars red, Budget bars blue.

6. Vercel Deployment

Local: `python scripts/export_for_web.py`
-> `git push` (includes `web/data/analysis.json`)
-> Vercel webhook builds `web/`
-> Live at `*.vercel.app`

Setting	Value
Root Directory	<code>web</code>
Framework	<code>Next.js</code>
Output Directory	leave empty (not out)
Build Command	<code>npm run build</code>

Update live data: run `export_for_web.py`, commit `web/data/` and `web/public/data/`, push. Do not redeploy failed old builds.

7. Quick Reference

Update website numbers

```
python scripts/export_for_web.py
# commit & push web/data/analysis.json
```

Regenerate Excel workbook

```
python scripts/export_excel_workbook.py
```

Run local analysis + PNG charts

```
pip install -r requirements.txt
python main.py
```

Analyze another ticker

```
export(ticker="AAPL") # in export_for_web.py
```

Local web preview

```
python scripts/export_for_web.py
cd web && npm install && npm run dev
```

8. Design Decisions

Decision	Rationale
Years as DataFrame index	Consistent <code>df.loc[year]</code> across modules
Static JSON for web	Simple Vercel hosting; refresh on push
Excel analysis uses formulas	Teaches FP&A modeling; assumptions drive outputs
<code>base_year = index[-2]</code>	Budget from prior year; variance vs latest year
<code>Net Income = OI * 0.75</code> in budget	Simplified tax, not actual effective rate
Sensitivity NI scaling	Preserves below-the-line ratio vs operating income
<code>main.py</code> vs export scripts	<code>main</code> = early 5-step demo; exports = full Corp Fin

9. File Map — What to Edit

Goal	File
Data source / year format	<code>src/data_fetcher.py</code>
Margins and growth	<code>src/pl_analysis.py</code>
Net income bridge	<code>src/margin_bridge.py</code>
FCF metrics	<code>src/cash_flow_analysis.py</code>
DSO / DIO / DPO / CCC	<code>src/working_capital.py</code>
ROE decomposition	<code>src/dupont_analysis.py</code>
Leverage ratios	<code>src/capital_structure.py</code>
Budget growth assumptions	<code>scripts/export_for_web.py</code> or Excel <code>15a_Assumptions</code>
Forecast method	<code>src/forecasting.py</code>
Sensitivity ranges	<code>src/sensitivity.py</code>
Summary narrative	<code>src/executive_summary.py</code>
Web JSON shape	<code>scripts/export_for_web.py</code> , <code>web/lib/types.ts</code>
Web charts	<code>web/components/charts.tsx</code>
Web layout	<code>web/app/page.tsx</code>
Excel formulas	<code>scripts/excel_formula_builder.py</code>
Excel sheet list	<code>scripts/export_excel_workbook.py</code>

10. Suggested Learning Path

1. Run `export_for_web.py` and read `web/data/analysis.json` alongside `web/lib/types.ts`
2. Open Excel `02_Income_Raw`, then `07_Margins` and inspect cell formulas
3. Read Python modules starting with `pl_analysis.py`; map fields to JSON
4. Trace `page.tsx` -> `charts.tsx` for the frontend
5. Change Excel `15a_Assumptions` or Python growth rates and compare outputs
6. Try `ticker="AAPL"` to verify the framework generalizes

Disclaimer

For education and portfolio use only. Data from Yahoo Finance may differ from official filings. Not financial advice.

Document version: 2026-06-18